FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

INFORME PROYECTO

Autores:
Garrido Mantilla, Yohan Javier (100%)
Rodríguez Iñigua, Fabricio Leonardo (100%)
Ordinola Rodríguez, Diego Miguel (100%)
Quiñonez Morón, Julio Marcelo (100%)

Curso:
Técnicas de programación orientada a objetos

Docente del Curso:
Mg. Jhon Paul Anampa García

Lima – Perú
2026-II

1. Descripción del problema

Situación problemática

En una biblioteca, llevar el control de los libros, usuarios, préstamos y devoluciones de manera manual (cuadernos, hojas sueltas o planillas aisladas) genera errores de registro y pérdida de información. Además, resulta difícil saber qué libros están disponibles, quién tiene un libro prestado y cuándo debe devolverlo.

Contexto

El problema se presenta en bibliotecas pequeñas y medianas (escolares, municipales o institucionales) que no cuentan con un sistema automatizado. Los antecedentes revisados muestran que este escenario es frecuente tanto en el Perú como en otros países de la región.

Personas o usuarios involucrados

- Bibliotecario o administrador: registra libros, usuarios, préstamos y devoluciones, y es quien más sufre la carga de trabajo manual.
- Usuarios o lectores: solicitan libros y esperan que la búsqueda y el préstamo sean rápidos y confiables.
- Institución: necesita información confiable sobre su inventario y el uso de sus libros.

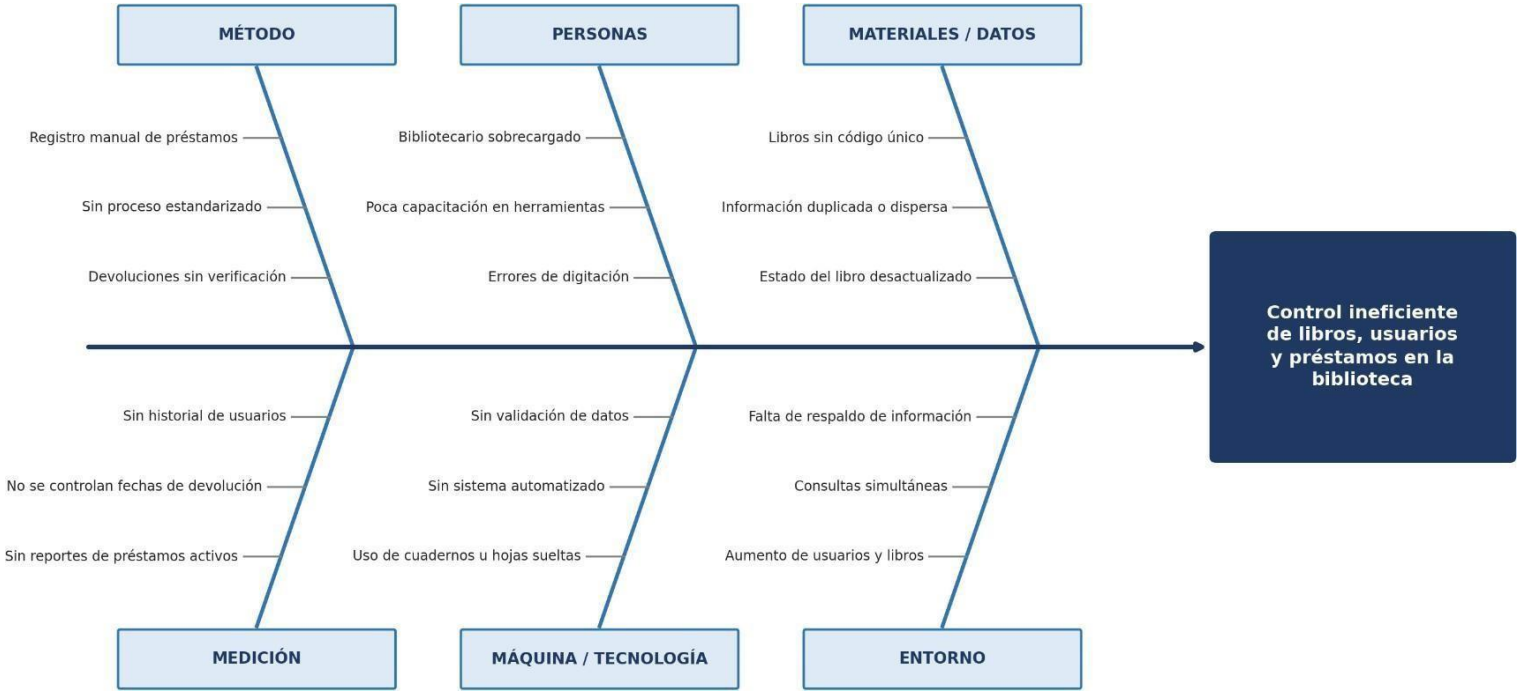
Consecuencias del problema

- Pérdida de tiempo al buscar libros, usuarios y registros de préstamo.
- Errores y duplicidad de datos; libros cuyo estado real no coincide con el registrado.
- Falta de trazabilidad: no se sabe con rapidez quién tiene un libro ni desde cuándo.
- Retrasos o pérdidas de libros por falta de control de devoluciones.
- Menor satisfacción de los usuarios y menor uso de la biblioteca.

2. Análisis de causas

Para identificar las causas se utilizó el diagrama de Ishikawa (Ishikawa, 1986) adaptado al contexto de software. El efecto analizado es el control ineficiente de libros, usuarios y préstamos en la biblioteca.

Diagrama de Ishikawa del problema de control de la biblioteca



Principales causas identificadas

- **Método:** el registro manual de préstamos y la ausencia de un proceso estandarizado provocan errores y devoluciones sin verificar.
- **Materiales / datos:** libros sin código único, información dispersa y estados desactualizados impiden saber qué está disponible.
- **Medición:** no existen reportes de préstamos activos ni control de fechas, por lo que no se detectan retrasos.
- **Máquina / tecnología:** el uso de cuadernos u hojas sueltas, sin validaciones, es la causa técnica principal que este proyecto busca resolver.
- **Personas y entorno:** la sobrecarga del bibliotecario y el crecimiento de
- usuarios y libros agravan el problema.

3. Antecedentes

La automatización del control de préstamos bibliotecarios ha sido abordada en varios trabajos que coinciden en que el manejo manual genera pérdida de tiempo e información y que un sistema informático mejora la gestión.

En el ámbito internacional, Matamoros y Sánchez (2011) diagnosticaron la biblioteca municipal del cantón Biblián (Ecuador) y encontraron que los préstamos se registraban a mano, lo que obligaba al usuario a esperar mientras se buscaba el material y disminuía su interés por asistir. Su aplicación web para el control de préstamos y consultas permitió a la bibliotecaria controlar mejor los libros y a los usuarios consultarlos en línea.

En el Perú, Ruiz Loro (2021) diseñó un sistema web con software libre para la biblioteca de la institución educativa Federico Villarreal (Piura). Su estudio halló que el 70 % de los encuestados no estaba satisfecho con el sistema vigente y que el 85
% consideraba necesario un sistema que optimizara los procesos.

Finalmente, Quispe-Lukaña y Ulloa-Gallardo (2024) implementaron un sistema web para optimizar el préstamo de libros en bibliotecas de Tambopata. Con un diseño preexperimental, reportaron que el tiempo de gestión bajó de 18,4 a 9,6 horas por proceso y que la percepción de eficiencia y facilidad de uso aumentó 45 %.

Estos trabajos confirman la vigencia del problema. A diferencia de ellos, que plantean soluciones web, este proyecto desarrolla un prototipo de consola en Java con Programación Orientada a Objetos.

Síntesis de los antecedentes revisados

Autor y año	Contexto	Solución	Resultado principal
Matamoros y Sánchez (2011)	Biblioteca municipal, Biblián (Ecuador)	Aplicación web de préstamos y consultas	Mejor control de libros y consulta en línea
Ruiz Loro (2021)	I. E. Federico Villarreal, Piura (Perú)	Diseño de sistema web con software libre	70 % insatisfecho; 85 % ve necesario un sistema
Quispe-Lukaña y Ulloa-Gallardo (2024)	Bibliotecas de Tambopata (Perú)	Sistema web de préstamos	Tiempo de 18,4 h a 9,6 h; +45 % eficiencia percibida

4. Objetivos del proyecto

Objetivo general (Mejorado)

Desarrollar un sistema de información en Java orientado a objetos, integrando una **Interfaz Gráfica de Usuario (GUI)**, que automatice la administración del inventario de libros, el registro de usuarios y el control de préstamos y devoluciones, reduciendo los errores del procesamiento manual y garantizando la trazabilidad de los ejemplares.

Objetivos específicos (Mejorado)

1. **Estructurar el Modelo POO:** Diseñar e implementar las clases de dominio fundamentales (Libro, Usuario, Préstamo y Biblioteca) aplicando principios de encapsulamiento, abstracción y composición.
2. **Construir la Interfaz Gráfica (GUI):** Diseñar ventanas gráficas interactivas para reemplazar la consola de comandos, facilitando la visualización del catálogo, la búsqueda intuitiva de libros y la captura accesible de datos.
3. **Gestión Interactiva de Inventario y Usuarios:** Implementar las operaciones CRUD para el catálogo de libros (búsqueda por código y por título insensible a mayúsculas/minúsculas) y para los usuarios del sistema.
4. **Automatización del Proceso de Préstamos y Devoluciones:** Integrar mecanismos que registren la transacción asociando un usuario y un libro con fecha automática (LocalDate.now()), actualizando dinámicamente el estado del libro (Disponible / Prestado).
5. **Robustez y Validación de Datos:** Aplicar manejo de excepciones personalizadas y sobrecarga de métodos para asegurar que las entradas por formularios visuales o campos de texto sean válidas y no afecten el flujo de ejecución.

5. Alcance del Sistema (Mejorado)

- **Módulo de Gestión de Libros**: Listado general en tablas o paneles con su estado de disponibilidad, búsqueda por código exacto y por título.

- **Módulo de Usuarios:** Registro de miembros o lectores y consulta de la lista de usuarios habilitados.

- **Módulo de préstamos y Devoluciones:** Proceso para generar transacciones asociando libro y usuario registro de la fecha del préstamo y actualización automática del libro a estado "Prestado" o "Disponible".

- **Interfaz Gráfica De Usuario (GUI):** Formularios interactivas de tablas, ventanas emergentes y botones de acción.

- **Control de Versiones y Trabajo Colaborativo:** Gestión modular del proyecto en GitHub con repositorio, ramas independientes por desarrollador e historial estructurado de integración mediante Pull Requests.

Consulta de inventario, consulta de usuarios, préstamo de libros y devolución de libros.

Aspectos fuera del proyecto

- Interfaz gráfica o versión web/móvil.
- Base de datos permanente: en esta etapa los datos se manejan en memoria.
- Multas, reservas, renovaciones, notificaciones y control de acceso por roles.
- Gestión de inventario avanzada (ISBN, categorías, múltiples ejemplares) y reportes estadísticos.

RESTRICCIONES IDENTIFICADAS

- Persistencia de datos en memoria volátil (Sin SGBD)

- Curva de aprendizaje e integración de Interfaz Gráfica

- Disponibilidad de información y tiempo de desarrollo reducido

- Validación de tipos de datos en la entrada del usuario

- Conflictos durante la integración de código en la GUI

CATEGORIA Y TIPO

- LIMITACION TECNICA
- CONOCIMIENTO TECNOLOGICO
- TIEMPO DE DESARROLLO
- RECURSOS O CALIDAD DEL SOFTWARE
- TRABAJO COLABORATIVO

IMPACTO EN EL PROYECTO

- Los datos registrados (libros usuarios, préstamos) se pierden al cerrar la aplicación Java
- Diseñar interfaces manuales sin un marco de trabajo ordenado puede ocasionar código acoplado y problemas de diseño.
- El cronograma académico restringe la ventana de tiempo para implementar la GUI y realizar pruebas completas
- Ingresos inválidos por campos de texto visuales pueden disparar excepciones no controladas
- (NumberForma tException, NullPointerException) deteniendo la GUI
- El trabajo en paralelo sobre componentes de la interfaz gráfica por diferentes integrantes puede acarrear colisiones o bloqueos al fusionar.

Alternativa o Estrategia de Afrontamiento

1. Implementar lectura y escritura de objetos en archivos locales mediante serialización de objetos Java) o almacenamiento estructurado en formato.json/csv.

2. Utilizar maquetadores visuales (como WindowBuilder), adoptar el patrón de diseño MVC (Modelo-Vista Controlador) para separar la lógica de negocio de la interfaz gráfica y reutilizar el código del modelo Java ya probado.

3. Aplicar metodologías ágiles asignando la migración de consolas a formularios GUI divididos por componentes entre los 4 integrantes del equipo mediante sus ramas de Git.

4. Implementar bloques try-catch robustos en la capa de la vista y aplicar

máscaras/validadores de texto (DocumentFilter) en los campos de entrada visuales antes de enviar la información al modelo.

5. Mantener la política rigurosa de control de versiones con ramas personales (Julio, Yohan, Diego, Fabrizio), integrar cambios únicamente mediante Pull Requests revisados por al menos 2 colaboradores y no trabajar directamente sobre la rama main.

6. Descripción de la solución propuesta

Elemento	Descripción
Nombre de la aplicación	Sistema de Gestión y Control de Préstamos de una Biblioteca
Usuarios principales	Bibliotecario o administrador (operador del sistema) y lectores (usuarios registrados que solicitan préstamos)
Funcionalidades iniciales	1. Listar libros; 2. Buscar libro por código; 3. Buscar libro por título; 4. Listar usuarios; 5. Realizar préstamo; 6. Devolver libro; 7. Listar préstamos; 8. Salir
Beneficios esperados	Menos errores de registro; disponibilidad de libros consultable al instante; trazabilidad de cada préstamo con usuario y fecha; ahorro de tiempo en búsquedas; base de código reutilizable para las siguientes entregas

La solución se construye con Programación Orientada a Objetos: cada entidad de la biblioteca es una clase con atributos y métodos propios, y el programa principal solo coordina el menú de consola.

5. Prototipo inicial de java

Clases implementadas

Clase	Responsabilidad
libro	Representa un libro: código, título, autor y disponibilidad (atributo privado disponible). Métodos prestar(), devolver() y toString(). Desarrollada por Julio
usuario	Representa a un miembro de la biblioteca: código y nombre, con sus métodos de consulta (getters). Desarrollada por Fabrizio y Julio
prestamo	Asocia un libro y un usuario con un código, la fecha automática (LocalDate.now()) y un estado activo/devuelto; método finalizarPrestamo(). Desarrollada por Yohan
biblioteca	Administra tres colecciones ArrayList (libros, usuarios y préstamos) y ejecuta las operaciones: agregar, buscar, listar, prestar y devolver. Desarrollada por Diego
App	Clase principal (main): crea los datos iniciales y muestra el menú de consola de 8 opciones

Aplicación de los conceptos vistos en clase

Concepto	Aplicación en el prototipo
Clases y objetos	Cuatro clases del modelo (libro, usuario, prestamo, biblioteca) y la clase App. En App se crean objetos: tres libros (El Principito, Don Quijote, Cien años de soledad) y dos usuarios (Julio y Carlos)
Métodos	agregarLibro(), agregarUsuario(), listarLibros(), listarUsuarios(), realizarPrestamo(), devolverLibro() y listarPrestamos() en biblioteca; prestar() y devolver() en libro; finalizarPrestamo() en prestamo
Sobrecarga de métodos	buscarLibro(int codigo) y buscarLibro(String titulo) en la clase biblioteca: el mismo nombre con distinto tipo de parámetro. La búsqueda por título usa equalsIgnoreCase, por lo que no distingue mayúsculas
Manejo de excepciones	En App, el menú está dentro de un bloque try-catch que captura NumberFormatException cuando el usuario escribe un texto en lugar de un número; el programa muestra «Error: debe ingresar un número válido.» y continúa
Colecciones	Tres ArrayList en biblioteca (libros, usuarios y préstamos), recorridas con for-each para buscar y listar

Concepto	Aplicación en el prototipo
Encapsulamiento	Atributos privados con métodos de acceso (getCodigo(), getTitulo(), isDisponible(), etc.) y toString() sobrescrito en cada clase

Funcionalidades desarrolladas y evidencias de ejecución

Las capturas muestran la ejecución desde la consola con las siguientes funcionalidades:

1. Consulta del inventario inicial (opción 1): los tres libros aparecen como Disponible.
2. Búsqueda por código (opción 2, código 3: Cien años de soledad) y por título (opción 3, «don quijote», que devuelve el libro de código 2 sin distinguir mayúsculas).
3. Consulta de usuarios (opción 4): Julio (código 1) y Carlos (código 2).
4. Préstamo (opción 5): préstamo 01, libro 2 y usuario 1; el sistema confirma «Préstamo realizado correctamente».
5. Cambio de estado: Don Quijote pasa de Disponible a Prestado, y la opción 7 muestra el préstamo activo con fecha 2026-09-27.
6. Devolución (opción 6, código 2): el sistema confirma «Libro devuelto correctamente» y el libro vuelve a Disponible. La opción 8 cierra el programa.

Ejecución del programa: menú, listado y búsquedas



Ejecución del programa: usuarios, préstamo y cambio de estado



Ejecución del programa: listado de préstamos, devolución y cierre



8. Evidencia de trabajo colaborativo

URL del repositorio

https://github.com/Juli0-Q/Proyecto_Java.git

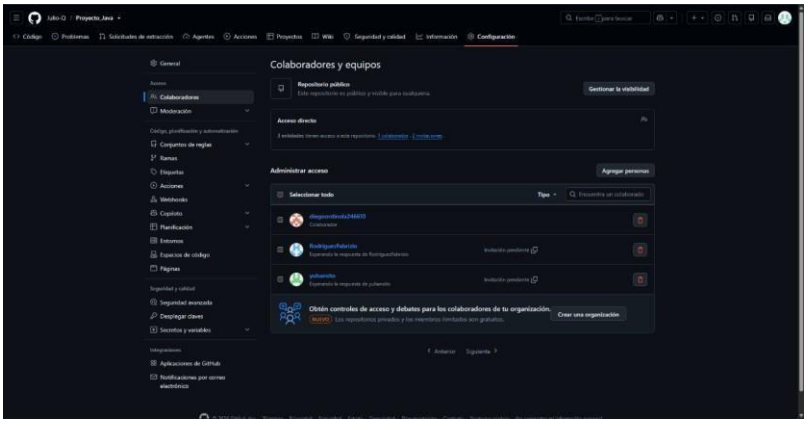
Integrantes y organización del trabajo

El repositorio se creó el 13 de septiembre de 2026 y el trabajo se organizó con una rama personal por integrante, más la rama main, que contiene la versión que funciona y donde nadie programa directamente.

Integrantes, usuarios de GitHub y clase asignada

Integrante	Usuario de GitHub	Rama	Clase a cargo
Garrido Mantilla, Yohan Javier	yuhansito	Yohan	prestamo
Ordinola Rodriguez, Diego Miguel	diegoordinola246610	Diego	biblioteca
Quiñonez Morón, Julio Marcelo	Juli0-Q (propietario)	Julio	libro
Rodriguez Iñigua, Fabrizio Leonardo	RodriguezFabrizio	Fabrizio	usuario

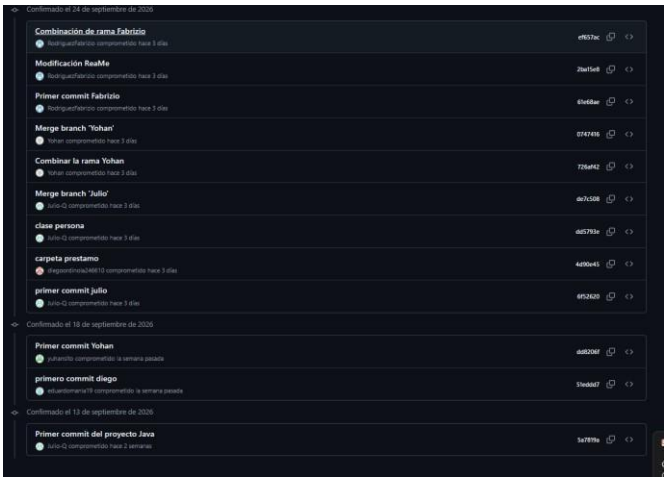
Colaboradores del repositorio (Configuración > Colaboradores)



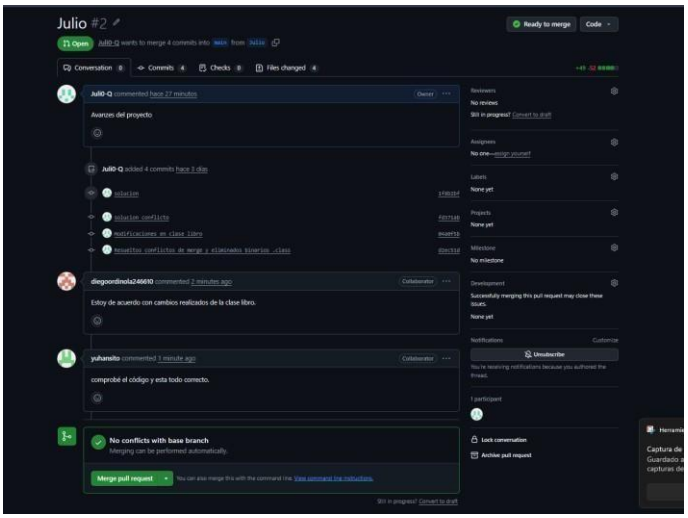
Historial de commits

El historial muestra commits de los cuatro integrantes, entre el 13 y el 24 de septiembre de 2026: «Primer commit del proyecto Java» y «primer commit julio» y «clase persona» (Julio); «Primer commit Yohan» y «Combinar la rama Yohan» (Yohan); «carpeta prestamo» (Diego); y «Primer commit Fabrizio», «Modificación ReMe» y «Combinación de rama Fabrizio» (Fabrizio).

Historial de commits con la participación de los cuatro integrantes



Pull request n.º 2 revisado por los colaboradores



Prácticas de control de versiones aplicadas

- **Ramas individuales y fusión (merge):** cada integrante trabajó en su rama y la integró a main.
- **Resolución de conflictos:** se resolvieron conflictos en README.md y en Libro.java revisando línea por línea.
- **Revisión de código con pull requests:** el pull request n.º 2 fue revisado por dos colaboradores antes de la fusión.
- **Repositorio limpio:** se eliminaron del control de versiones los archivos compilados (.class) y se recomienda excluir /bin/ con .gitignore.
- **Git stash:** el equipo investigó su uso para guardar cambios sin confirmar al cambiar de rama.

Requerimientos funcionales

2.1 Módulo de Gestión de Libros

RF01 – Registrar libro

- Descripción: El sistema debe permitir registrar un libro mediante un formulario con código, título y autor. Al registrarse, el libro queda con estado "Disponible".
- Prioridad: Alta
- OE: 1, 3

RF02 – Validar código único de libro

- Descripción: El sistema no debe permitir registrar dos libros con el mismo código. Si el código ya existe, debe mostrar un mensaje de error y no guardar el registro.
- Prioridad: Alta
- OE: 3, 5

RF03 – Listar libros con su estado

- Descripción: El sistema debe mostrar en una tabla todos los libros registrados con código, título, autor y estado de disponibilidad.
- Prioridad: Alta
- OE: 2, 3

RF04 – Buscar libro por código

- Descripción: El sistema debe permitir buscar un libro ingresando su código exacto y mostrar sus datos y su estado. Si no existe, debe informarlo.
- Prioridad: Alta
- OE: 3

RF05 – Buscar libro por título

- Descripción: El sistema debe permitir buscar libros por título sin distinguir mayúsculas de minúsculas (por ejemplo, "don quijote" encuentra "Don Quijote").
- Prioridad: Alta
- OE: 3

RF06 – Modificar datos de un libro

- Descripción: El sistema debe permitir editar el título y el autor de un libro existente. El código no puede modificarse.
- Prioridad: Media
- OE: 3

RF07 – Eliminar libro

- Descripción: El sistema debe permitir eliminar un libro del catálogo, siempre que no tenga un préstamo activo. Si está prestado, debe rechazar la operación con un mensaje.
- Prioridad: Media
- OE: 3, 5

2.2 Módulo de Usuarios

RF08 – Registrar usuario

- Descripción: El sistema debe permitir registrar un lector con código y nombre mediante un formulario.
- Prioridad: Alta
- OE: 1, 3

RF09 – Validar código único de usuario

- Descripción: El sistema no debe permitir registrar dos usuarios con el mismo código y debe mostrar un mensaje de error en ese caso.
- Prioridad: Alta
- OE: 3, 5

RF10 – Listar usuarios

- Descripción: El sistema debe mostrar en una tabla la lista de usuarios registrados con su código y nombre.
- Prioridad: Alta
- OE: 2, 3

RF11 – Buscar usuario por código

- Descripción: El sistema debe permitir consultar un usuario por su código y mostrar sus datos. Si no existe, debe informarlo.
- Prioridad: Media
- OE: 3

RF12 – Modificar datos de un usuario

- Descripción: El sistema debe permitir editar el nombre de un usuario registrado. El código no puede modificarse.
- Prioridad: Media
- OE: 3

RF13 – Eliminar usuario

- Descripción: El sistema debe permitir eliminar un usuario siempre que no tenga préstamos activos. En caso contrario, debe rechazar la operación con un mensaje.
- Prioridad: Media
- OE: 3, 5

2.3 Módulo de Préstamos y Devoluciones

RF14 – Registrar préstamo

- Descripción: El sistema debe permitir registrar un préstamo asociando un usuario y un libro, con un código de préstamo. La fecha se asigna automáticamente con la fecha actual del sistema.
- Prioridad: Alta
- OE: 4

RF15 – Impedir el préstamo de un libro no disponible

- Descripción: El sistema no debe permitir prestar un libro cuyo estado sea "Prestado". Debe mostrar un mensaje indicando que el libro no está disponible.
- Prioridad: Alta
- OE: 4, 5

RF16 – Verificar existencia de libro y usuario en el préstamo

- Descripción: Antes de registrar un préstamo, el sistema debe verificar que el libro y el usuario existan. Si alguno no existe, debe cancelar la operación e informar el motivo.
- Prioridad: Alta
- OE: 4, 5

RF17 – Actualizar el estado del libro al prestar

- Descripción: Al registrarse un préstamo, el sistema debe cambiar automáticamente el estado del libro de "Disponible" a "Prestado".
- Prioridad: Alta
- OE: 4

RF18 – Registrar devolución

- Descripción: El sistema debe permitir registrar la devolución de un libro prestado. El préstamo asociado pasa de estado "Activo" a "Devuelto".
- Prioridad: Alta
- OE: 4

RF19 – Actualizar el estado del libro al devolver

- Descripción: Al registrarse una devolución, el sistema debe cambiar automáticamente el estado del libro de "Prestado" a "Disponible".
- Prioridad: Alta
- OE: 4

RF20 – Listar préstamos

- Descripción: El sistema debe mostrar todos los préstamos con código, libro, usuario, fecha y estado (Activo/Devuelto).
- Prioridad: Alta
- OE: 2, 4

RF21 – Listar préstamos activos

- Descripción: El sistema debe permitir filtrar y mostrar solo los préstamos que aún no han sido devueltos.
- Prioridad: Media
- OE: 4

RF22 – Consultar quién tiene un libro prestado

- Descripción: Dado un libro con estado "Prestado", el sistema debe mostrar el usuario que lo tiene y la fecha desde la cual lo posee, para garantizar la trazabilidad del ejemplar.
- Prioridad: Media
- OE: 4

RF23 – Consultar historial de préstamos de un usuario

- Descripción: El sistema debe mostrar todos los préstamos (activos y devueltos) asociados a un usuario determinado.
- Prioridad: Media
- OE: 4

2.4 Interfaz gráfica, validación y persistencia

RF24 – Ventana principal con navegación por módulos

- Descripción: El sistema debe contar con una ventana principal que permita acceder a los módulos de Libros, Usuarios y Préstamos/Devoluciones mediante menú o botones, y salir de la aplicación de forma controlada.
- Prioridad: Alta
- OE: 2

RF25 – Validación de datos de entrada en formularios

- Descripción: El sistema debe validar los campos de los formularios antes de enviar la información al modelo: campos obligatorios no vacíos y campos numéricos (códigos) con solo dígitos. Ante datos inválidos, debe mostrar un mensaje claro y no ejecutar la operación.
- Prioridad: Alta
- OE: 5

RF26 – Manejo de excepciones personalizadas

- Descripción: El sistema debe controlar los errores de negocio (código duplicado, libro no encontrado, libro no disponible, entrada inválida) mediante excepciones personalizadas, mostrándolos al usuario en ventanas emergentes sin detener la aplicación.
- Prioridad: Alta
- OE: 5

RF27 – Confirmación de acciones críticas

- Descripción: El sistema debe solicitar confirmación al usuario mediante una ventana emergente antes de eliminar un libro o un usuario y antes de registrar una devolución.
- Prioridad: Baja
- OE: 2, 5

RF28 – Carga de datos iniciales

- Descripción: Al iniciar, el sistema debe cargar un conjunto de datos de ejemplo (libros y usuarios) o los datos guardados previamente, de modo que pueda utilizarse inmediatamente.
- Prioridad: Media
- OE: 1, 3

RF29 – Guardar y cargar datos en archivo local

- Descripción: El sistema debe permitir guardar los libros, usuarios y préstamos en un archivo local (serialización de objetos Java o formato JSON/CSV) y cargarlos al iniciar, para que la información no se pierda al cerrar la aplicación. Corresponde a la estrategia de afrontamiento de la restricción de persistencia en memoria volátil, sin utilizar un SGBD.
- Prioridad: Media
- OE: 3

3. Historias de usuario

Formato: Como [tipo de usuario], quiero [funcionalidad], para [beneficio].

3.1 Módulo de Gestión de Libros

- HU01 (RF01): Como bibliotecario, quiero registrar un libro con su código, título y autor, para mantener el catálogo de la biblioteca actualizado.

  HU02 (RF02): Como bibliotecario, quiero que el sistema rechace libros con un código repetido, para evitar duplicidad de datos en el inventario.

- HU03 (RF03): Como bibliotecario, quiero ver en una tabla todos los libros con su estado, para saber de inmediato cuáles están disponibles.

- HU04 (RF04): Como bibliotecario, quiero buscar un libro por su código exacto, para atender rápidamente a un lector que conoce el código.

- HU05 (RF05): Como bibliotecario, quiero buscar un libro por su título sin importar mayúsculas o minúsculas, para encontrarlo, aunque el título se escriba de forma distinta.

- HU06 (RF06): Como bibliotecario, quiero modificar el título o el autor de un libro, para corregir errores de registro.

- HU07 (RF07): Como bibliotecario, quiero eliminar un libro del catálogo solo si no está prestado, para retirar libros dados de baja sin perder el control de los préstamos vigentes.

3.2 Módulo de Gestión de Usuarios

- HU08 (RF08): Como bibliotecario, quiero registrar a un lector con su código y nombre, para que pueda solicitar préstamos.

- HU09 (RF09): Como bibliotecario, quiero que el sistema impida registrar dos usuarios con el mismo código, para identificar a cada lector de forma única.

- HU10 (RF10): Como bibliotecario, quiero ver la lista de usuarios registrados, para confirmar quiénes están habilitados para pedir libros.

- HU11 (RF11): Como bibliotecario, quiero buscar un usuario por su código, para verificar sus datos antes de registrar un préstamo.

- HU12 (RF12): Como bibliotecario, quiero modificar el nombre de un usuario, para mantener sus datos correctos.

- HU13 (RF13): Como bibliotecario, quiero eliminar a un usuario solo si no tiene préstamos activos, para no perder la trazabilidad de los libros que tiene en su poder.

3.3 Módulo de Préstamos y Devoluciones

- HU14 (RF14): Como bibliotecario, quiero registrar un préstamo asociando un lector con un libro y con la fecha automática, para dejar constancia de quién se lleva cada libro y desde cuándo.

- HU15 (RF15): Como bibliotecario, quiero que el sistema no permita prestar un libro que ya está prestado, para evitar préstamos duplicados del mismo ejemplar.

- HU16 (RF16): Como bibliotecario, quiero que el sistema verifique que el libro y el usuario existen antes de prestar, para evitar préstamos con datos inválidos.

- HU17 (RF17): Como bibliotecario, quiero que el libro pase a "Prestado" automáticamente al registrar el préstamo, para que el catálogo refleje siempre su estado real.

- HU18 (RF18): Como bibliotecario, quiero registrar la devolución de un libro, para cerrar el préstamo y tener el control de las devoluciones.

- HU19 (RF19): Como bibliotecario, quiero que el libro vuelva a "Disponible" automáticamente al devolverse, para que otros lectores puedan solicitarlo.

- HU20 (RF20): Como bibliotecario, quiero ver el listado de todos los préstamos con su fecha y estado, para tener el registro completo de la actividad de la biblioteca.

- HU21 (RF21): Como bibliotecario, quiero ver solo los préstamos activos, para identificar qué libros aún no han sido devueltos.

- HU22 (RF22): Como bibliotecario, quiero saber quién tiene un libro prestado y desde cuándo, para ubicarlo y gestionar su devolución.

- HU23 (RF23): Como lector, quiero que el bibliotecario pueda consultar el historial de mis préstamos, para conocer qué libros he solicitado y cuáles debo devolver.

3.4 Interfaz gráfica, validación y persistencia

- HU24 (RF24): Como bibliotecario, quiero una ventana principal con acceso claro a los módulos de libros, usuarios y préstamos, para trabajar de forma intuitiva sin usar la consola.

- HU25 (RF25): Como bibliotecario, quiero que los formularios validen los datos que escribo, para evitar errores por campos vacíos o valores no numéricos.

- HU26 (RF26): Como bibliotecario, quiero recibir mensajes claros cuando ocurre un error, para corregirlo sin que el programa se cierre.

- HU27 (RF27): Como bibliotecario, quiero una confirmación antes de eliminar o devolver, para evitar acciones accidentales.

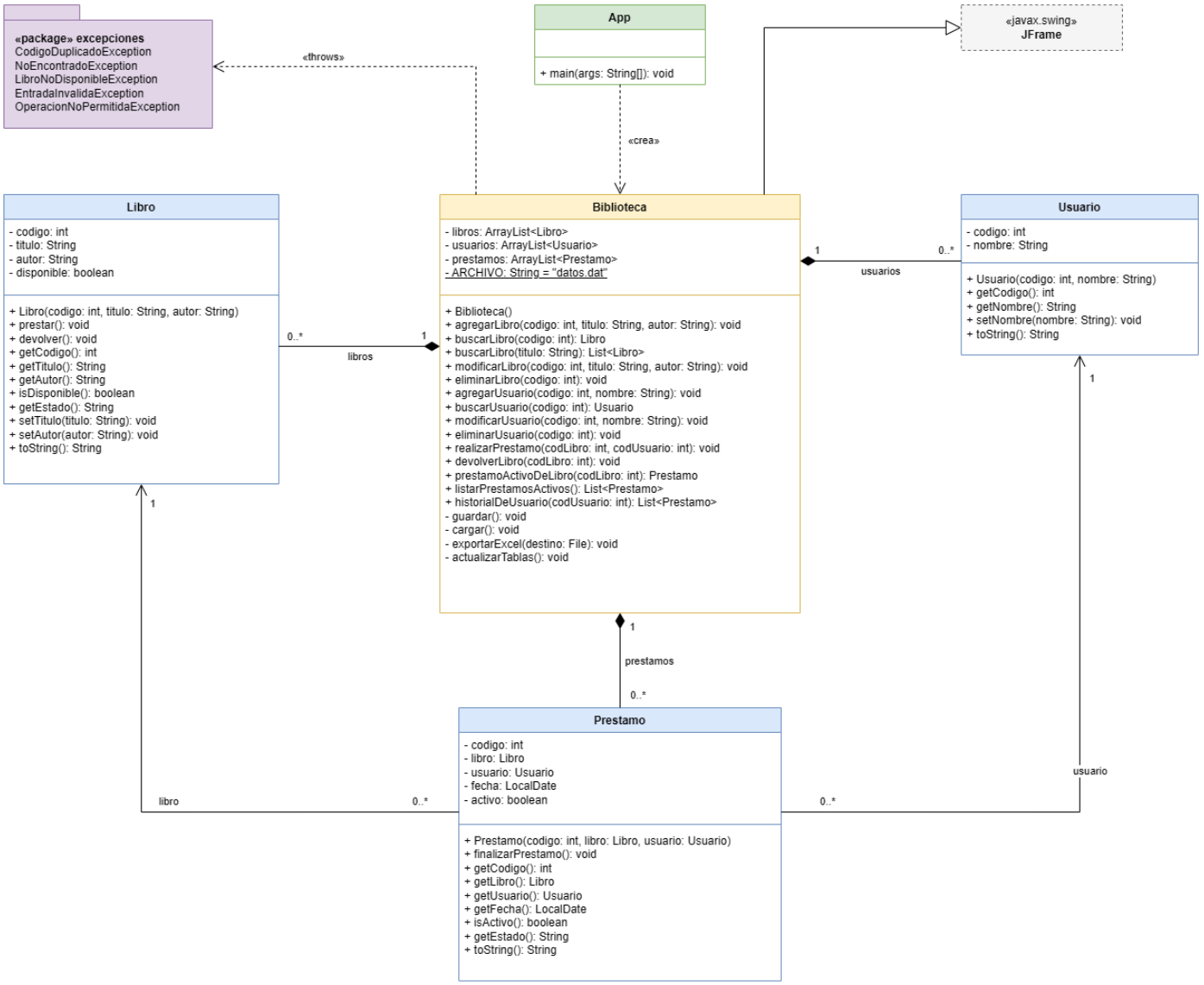
- HU28 (RF28): Como bibliotecario, quiero que el sistema inicie con datos de ejemplo o con los datos guardados, para poder comenzar a trabajar de inmediato.

- HU29 (RF29): Como administrador de la institución, quiero que el sistema guarde y recupere los datos desde un archivo local, para no perder el inventario ni los préstamos al cerrar la aplicación.

Diagrama de clases:

UPN
UNIVERSIDAD PRIVADA DEL NORTE

Diagrama de clases - Sistema de Gestión y Control de Préstamos de una Biblioteca



Código y ejecución de programa – Mejora de proyecto:

```
/**
 * Clase principal del sistema. Tiene tres partes:
 * 1) OPERACIONES: las listas de libros, usuarios y préstamos, y lo que se puede hacer con ellas.
 * 2) ARCHIVO: guarda los datos en un archivo y los exporta a Excel.
 * 3) VENTANA: la interfaz gráfica con la que se registran los datos (por eso extiende JFrame).
 */
public class Biblioteca extends JFrame {

    // ================================================================== 1) OPERACIONES

    /** Todos los libros del catálogo. */
    private ArrayList<Libro> libros = new ArrayList<>();
    /** Todos los usuarios registrados. */
    private ArrayList<Usuario> usuarios = new ArrayList<>();
    /** Todos los préstamos, activos y devueltos. */
    private ArrayList<Prestamo> prestamos = new ArrayList<>();

    // ---- Libros

    /**
     * Registra un libro nuevo.
     *
     * @param codigo código único del libro
     * @param titulo título del libro
     * @param autor autor del libro
     * @throws CodigoDuplicadoException si ya existe un libro con ese código
     */
    public void agregarLibro(int codigo, String titulo, String autor) throws
CodigoDuplicadoException {
        for (Libro l : libros) {
            if (l.getCodigo() == codigo) {
                throw new CodigoDuplicadoException("Ya existe un libro con el código " + codigo +
".");
            }
        }
        libros.add(new Libro(codigo, titulo, autor));
    }

    /**
     * Busca un libro por su código exacto.
     *
     * @param codigo código a buscar
     * @return el libro encontrado
     * @throws NoEncontradoException si no existe ningún libro con ese código
     */
    public Libro buscarLibro(int codigo) throws NoEncontradoException {
        for (Libro l : libros) {
            if (l.getCodigo() == codigo) {
                return l;
            }
```

```
        }
        throw new NoEncontradoException("No existe un libro con el código " + codigo + ".");
    }

    /**
     * Sobrecarga de buscarLibro: busca por título, sin distinguir mayúsculas de minúsculas.
     * Basta con escribir una parte del título ("quijote" encuentra "Don Quijote").
     *
     * @param titulo texto a buscar dentro de los títulos
     * @return los libros que coinciden (lista vacía si no hay ninguno)
     */
    public List<Libro> buscarLibro(String titulo) {
        List<Libro> encontrados = new ArrayList<>();
        for (Libro l : libros) {
            if (l.getTitulo().toLowerCase().contains(titulo.toLowerCase())) {
                encontrados.add(l);
            }
        }
        return encontrados;
    }

    /**
     * Cambia el título y el autor de un libro. El código no se puede cambiar.
     *
     * @throws NoEncontradoException si no existe un libro con ese código
     */
    public void modificarLibro(int codigo, String titulo, String autor) throws
NoEncontradoException {
        Libro libro = buscarLibro(codigo);
        libro.setTitulo(titulo);
        libro.setAutor(autor);
    }

    /**
     * Elimina un libro del catálogo.
     *
     * @throws NoEncontradoException si no existe un libro con ese código
     * @throws OperacionNoPermitidaException si el libro está prestado
     */
    public void eliminarLibro(int codigo) throws NoEncontradoException,
OperacionNoPermitidaException {
        Libro libro = buscarLibro(codigo);
        if (!libro.isDisponible()) {
            throw new OperacionNoPermitidaException("No se puede eliminar un libro que está
prestado.");
        }
        libros.remove(libro);
    }

    // ---- Usuarios

    /**
     * Registra un usuario nuevo.
```

```
     *
     * @param codigo código único del usuario
     * @param nombre nombre del lector
     * @throws CodigoDuplicadoException si ya existe un usuario con ese código
     */
    public void agregarUsuario(int codigo, String nombre) throws CodigoDuplicadoException {
        for (Usuario u : usuarios) {
            if (u.getCodigo() == codigo) {
                throw new CodigoDuplicadoException("Ya existe un usuario con el código " + codigo +
".");
            }
        }
        usuarios.add(new Usuario(codigo, nombre));
    }

    /**
     * Busca un usuario por su código.
     *
     * @throws NoEncontradoException si no existe
     */
    public Usuario buscarUsuario(int codigo) throws NoEncontradoException {
        for (Usuario u : usuarios) {
            if (u.getCodigo() == codigo) {
                return u;
            }
        }
        throw new NoEncontradoException("No existe un usuario con el código " + codigo + ".");
    }

    /**
     * Cambia el nombre de un usuario. El código no se puede cambiar.
     *
     * @throws NoEncontradoException si no existe
     */
    public void modificarUsuario(int codigo, String nombre) throws NoEncontradoException {
        buscarUsuario(codigo).setNombre(nombre);
    }

    /**
     * Elimina un usuario.
     *
     * @throws NoEncontradoException si no existe
     * @throws OperacionNoPermitidaException si el usuario tiene libros sin devolver
     */
    public void eliminarUsuario(int codigo) throws NoEncontradoException,
OperacionNoPermitidaException {
        Usuario usuario = buscarUsuario(codigo);
        for (Prestamo p : prestamos) {
            if (p.isActivo() && p.getUsuario() == usuario) {
                throw new OperacionNoPermitidaException("No se puede eliminar un usuario que tiene
préstamos activos.");
            }
        }
```

```
        usuarios.remove(usuario);
    }

    // ---- Préstamos

    /**
     * Registra un préstamo. El código del préstamo se asigna solo y la fecha es la de hoy.
     *
     * @param codLibro código del libro que se presta
     * @param codUsuario código del usuario que lo recibe
     * @throws NoEncontradoException si el libro o el usuario no existen
     * @throws LibroNoDisponibleException si el libro ya está prestado
     */
    public void realizarPrestamo(int codLibro, int codUsuario) throws NoEncontradoException,
LibroNoDisponibleException {
        Libro libro = buscarLibro(codLibro);         // falla si el libro no existe
        Usuario usuario = buscarUsuario(codUsuario); // falla si el usuario no existe
        libro.prestar();                              // falla si el libro ya está prestado
        prestamos.add(new Prestamo(prestamos.size() + 1, libro, usuario));
    }

    /**
     * Registra la devolución de un libro: cierra su préstamo y el libro vuelve a estar disponible.
     *
     * @param codLibro código del libro que se devuelve
     * @throws NoEncontradoException si el libro no existe o no está prestado
     */
    public void devolverLibro(int codLibro) throws NoEncontradoException {
        prestamoActivoDeLibro(codLibro).finalizarPrestamo();
    }

    /**
     * Busca el préstamo activo de un libro. Sirve para saber quién lo tiene y desde cuándo.
     *
     * @param codLibro código del libro
     * @return el préstamo activo de ese libro
     * @throws NoEncontradoException si el libro no existe o no está prestado
     */
    public Prestamo prestamoActivoDeLibro(int codLibro) throws NoEncontradoException {
        Libro libro = buscarLibro(codLibro);
        for (Prestamo p : prestamos) {
            if (p.isActivo() && p.getLibro() == libro) {
                return p;
            }
        }
        throw new NoEncontradoException("El libro \"" + libro.getTitulo() + "\" no está
prestado.");
    }

    /**
     * @return solo los préstamos que aún no se han devuelto
     */
    public List<Prestamo> listarPrestamosActivos() {
```

```
        List<Prestamo> activos = new ArrayList<>();
        for (Prestamo p : prestamos) {
            if (p.isActivo()) {
                activos.add(p);
            }
        }
        return activos;
    }

    /**
     * Obtiene todos los préstamos (activos y devueltos) de un usuario.
     *
     * @param codUsuario código del usuario
     * @throws NoEncontradoException si el usuario no existe
     */
    public List<Prestamo> historialDeUsuario(int codUsuario) throws NoEncontradoException {
        Usuario usuario = buscarUsuario(codUsuario);
        List<Prestamo> historial = new ArrayList<>();
        for (Prestamo p : prestamos) {
            if (p.getUsuario() == usuario) {
                historial.add(p);
            }
        }
        return historial;
    }

    // ========================================================================= 2) ARCHIVO

    /** Nombre del archivo donde se guardan los datos. */
    private static final String ARCHIVO = "datos.dat";

    /**
     * Guarda las tres listas en el archivo. Se llama después de cada cambio, así no se pierde nada
al cerrar.
     */
    private void guardar() {
        try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(ARCHIVO))) {
            out.writeObject(libros);
            out.writeObject(usuarios);
            out.writeObject(prestamos);
        } catch (IOException e) {
            JOptionPane.showMessageDialog(this, "No se pudieron guardar los datos: " +
e.getMessage(),
                    "Error", JOptionPane.ERROR_MESSAGE);
        }
    }

    /**
     * Lee las tres listas desde el archivo al iniciar el programa.
     * Si el archivo no existe, usa datos de ejemplo; si está dañado, lo aparta y usa datos de
ejemplo.
     */
    @SuppressWarnings("unchecked")
```

```
    private void cargar() {
        File archivo = new File(ARCHIVO);
        if (!archivo.exists()) {
            datosEjemplo();
            return;
        }
        try (ObjectInputStream in = new ObjectInputStream(new FileInputStream(archivo))) {
            libros = (ArrayList<Libro>) in.readObject();
            usuarios = (ArrayList<Usuario>) in.readObject();
            prestamos = (ArrayList<Prestamo>) in.readObject();
        } catch (IOException | ClassNotFoundException e) {
            // El archivo está dañado: se aparta (no se pierde) y se empieza con datos de ejemplo.
            File apartado = new File("datos_con_error.dat");
            apartado.delete();
            archivo.renameTo(apartado);
            datosEjemplo();
            JOptionPane.showMessageDialog(null, "No se pudo leer " + ARCHIVO
                    + ".\nSe guardó como datos_con_error.dat y se usarán datos de ejemplo.");
        }
    }

    /** Crea 3 libros y 2 usuarios de ejemplo para poder usar el sistema la primera vez. */
    private void datosEjemplo() {
        libros = new ArrayList<>();
        usuarios = new ArrayList<>();
        prestamos = new ArrayList<>();
        libros.add(new Libro(1, "El Principito", "Antoine de Saint-Exupéry"));
        libros.add(new Libro(2, "Don Quijote", "Miguel de Cervantes"));
        libros.add(new Libro(3, "Cien años de soledad", "Gabriel García Márquez"));
        usuarios.add(new Usuario(1, "Julio"));
        usuarios.add(new Usuario(2, "Carlos"));
    }

    /**
     * Exporta todos los datos a un archivo .csv que se abre con Excel.
     * Tiene tres bloques: LIBROS, USUARIOS y PRÉSTAMOS.
     *
     * @param destino archivo que se va a crear
     * @throws IOException si no se puede escribir el archivo
     */
    private void exportarExcel(File destino) throws IOException {
        try (PrintWriter out = new PrintWriter(destino, "UTF-8")) {
            out.print('\uFEFF'); // marca para que Excel muestre bien las tildes y la ñ

            out.println("LIBROS");
            out.println(fila("Código", "Título", "Autor", "Estado"));
            for (Libro l : libros) {
                out.println(fila(l.getCodigo(), l.getTitulo(), l.getAutor(), l.getEstado()));
            }

            out.println();
            out.println("USUARIOS");
            out.println(fila("Código", "Nombre"));
```

```
            for (Usuario u : usuarios) {
                out.println(fila(u.getCodigo(), u.getNombre()));
            }

            out.println();
            out.println("PRÉSTAMOS");
            out.println(fila("Código", "Libro", "Usuario", "Fecha", "Estado"));
            for (Prestamo p : prestamos) {
                out.println(fila(p.getCodigo(), p.getLibro().getTitulo(),
p.getUsuario().getNombre(),
                        p.getFecha(), p.getEstado()));
            }
        }
    }

    /**
     * Arma una fila del archivo CSV: cada dato entre comillas y separado por comas.
     *
     * @param datos los valores de la fila
     * @return la fila lista para escribir
     */
    private String fila(Object... datos) {
        StringBuilder linea = new StringBuilder();
        for (int i = 0; i < datos.length; i++) {
            if (i > 0) {
                linea.append(",");
            }
            linea.append("\"").append(datos[i].toString().replace("\"", "\"\"")).append("\"");
        }
        return linea.toString();
    }

    // ============================================================================ 3) VENTANA

    // Tablas y campos de texto de cada pestaña
    private DefaultTableModel modeloLibros = crearModelo("Código", "Título", "Autor", "Estado");
    private JTable tablaLibros = new JTable(modeloLibros);
    private JTextField txtBuscarCodigo = new JTextField(6);
    private JTextField txtBuscarTitulo = new JTextField(12);
    private JTextField txtLibroCodigo = new JTextField(6);
    private JTextField txtLibroTitulo = new JTextField(16);
    private JTextField txtLibroAutor = new JTextField(16);

    private DefaultTableModel modeloUsuarios = crearModelo("Código", "Nombre");
    private JTable tablaUsuarios = new JTable(modeloUsuarios);
    private JTextField txtBuscarUsuario = new JTextField(6);
    private JTextField txtUsuarioCodigo = new JTextField(6);
    private JTextField txtUsuarioNombre = new JTextField(20);

    private DefaultTableModel modeloPrestamos =
            crearModelo("Préstamo", "Cód. libro", "Libro", "Cód. usuario", "Usuario", "Fecha",
"Estado");
    private JTable tablaPrestamos = new JTable(modeloPrestamos);
```

```
    private JCheckBox chkActivos = new JCheckBox("Mostrar solo préstamos activos");
    private JTextField txtPrestLibro = new JTextField(6);
    private JTextField txtPrestUsuario = new JTextField(6);

    /**
     * Representa lo que hace un botón. Puede lanzar cualquiera de nuestras excepciones,
     * que el método boton() captura para mostrar el error en una ventana.
     */
    private interface Accion {
        void ejecutar() throws Exception;
    }

    /** Construye la ventana: carga los datos, crea las tres pestañas y el menú, y llena las
tablas. */
    public Biblioteca() {
        super("Sistema de Gestión y Control de Préstamos de una Biblioteca");
        cargar();

        JTabbedPane pestanas = new JTabbedPane();
        pestanas.addTab("Libros", crearPestanaLibros());
        pestanas.addTab("Usuarios", crearPestanaUsuarios());
        pestanas.addTab("Préstamos y devoluciones", crearPestanaPrestamos());
        add(pestanas);
        setJMenuBar(crearMenu());

        actualizarTablas();
        setSize(900, 560);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
    }

    // ---- Menú: Archivo > Exportar a Excel / Salir

    /** Crea el menú Archivo, con las opciones Exportar a Excel y Salir. */
    private JMenuBar crearMenu() {
        JMenuItem exportar = new JMenuItem("Exportar a Excel...");
        exportar.addActionListener(e -> elegirDestinoYExportar());
        JMenuItem salir = new JMenuItem("Salir");
        salir.addActionListener(e -> {
            if (confirmar("¿Desea salir del sistema?")) {
                System.exit(0);
            }
        });
        JMenu menuArchivo = new JMenu("Archivo");
        menuArchivo.add(exportar);
        menuArchivo.add(salir);
        JMenuBar barra = new JMenuBar();
        barra.add(menuArchivo);
        return barra;
    }

    /** Pregunta dónde guardar el archivo y lo exporta a Excel. Muestra un mensaje de éxito o de
error. */
```

```
    private void elegirDestinoYExportar() {
        JFileChooser selector = new JFileChooser();
        selector.setSelectedFile(new File("biblioteca.csv"));
        if (selector.showSaveDialog(this) != JFileChooser.APPROVE_OPTION) {
            return;
        }
        File destino = selector.getSelectedFile();
        if (!destino.getName().toLowerCase().endsWith(".csv")) {
            destino = new File(destino.getPath() + ".csv");
        }
        try {
            exportarExcel(destino);
            JOptionPane.showMessageDialog(this, "Archivo exportado en:\n" +
destino.getAbsolutePath()
                    + "\nÁbralo con Excel.");
        } catch (IOException e) {
            mostrarError("No se pudo exportar: " + e.getMessage()
                    + "\n(Si el archivo está abierto en Excel, ciérrelo.)");
        }
    }

    // ---- Pestaña Libros

    /** Crea la pestaña de libros: buscadores arriba, tabla en el centro y formulario con botones
abajo. */
    private JPanel crearPestanaLibros() {
        // Arriba: búsquedas
        JPanel arriba = new JPanel();
        arriba.add(new JLabel("Código:"));
        arriba.add(txtBuscarCodigo);
        arriba.add(boton("Buscar por código", () -> {
            Libro libro = buscarLibro(numero(txtBuscarCodigo, "Código"));
            llenarLibros(Arrays.asList(libro));
        }));
        arriba.add(new JLabel("   Título:"));
        arriba.add(txtBuscarTitulo);
        arriba.add(boton("Buscar por título", () -> {
            List<Libro> encontrados = buscarLibro(texto(txtBuscarTitulo, "Título"));
            if (encontrados.isEmpty()) {
                throw new NoEncontradoException("No se encontraron libros con ese título.");
            }
            llenarLibros(encontrados);
        }));
        arriba.add(boton("Mostrar todos", () -> actualizarTablas()));

        // Al hacer clic en una fila, sus datos pasan al formulario
        tablaLibros.getSelectionModel().addListSelectionListener(e -> {
            int fila = tablaLibros.getSelectedRow();
            if (fila >= 0) {
                txtLibroCodigo.setText(modeloLibros.getValueAt(fila, 0).toString());
                txtLibroTitulo.setText(modeloLibros.getValueAt(fila, 1).toString());
                txtLibroAutor.setText(modeloLibros.getValueAt(fila, 2).toString());
            }
```

```
        });

        // Abajo: formulario y botones
        JPanel campos = new JPanel();
        campos.add(new JLabel("Código:"));
        campos.add(txtLibroCodigo);
        campos.add(new JLabel("Título:"));
        campos.add(txtLibroTitulo);
        campos.add(new JLabel("Autor:"));
        campos.add(txtLibroAutor);

        JPanel botones = new JPanel();
        botones.add(boton("Registrar", () -> {
            agregarLibro(numero(txtLibroCodigo, "Código"), texto(txtLibroTitulo, "Título"),
texto(txtLibroAutor, "Autor"));
            limpiar(txtLibroCodigo, txtLibroTitulo, txtLibroAutor);
            cambioRealizado("Libro registrado correctamente.");
        }));
        botones.add(boton("Modificar", () -> {
            modificarLibro(numero(txtLibroCodigo, "Código"), texto(txtLibroTitulo, "Título"),
texto(txtLibroAutor, "Autor"));
            limpiar(txtLibroCodigo, txtLibroTitulo, txtLibroAutor);
            cambioRealizado("Libro modificado correctamente.");
        }));
        botones.add(boton("Eliminar", () -> {
            int codigo = numero(txtLibroCodigo, "Código");
            buscarLibro(codigo); // verifica que exista
            if (!confirmar("¿Eliminar el libro con código " + codigo + "?")) {
                return;
            }
            eliminarLibro(codigo);
            limpiar(txtLibroCodigo, txtLibroTitulo, txtLibroAutor);
            cambioRealizado("Libro eliminado correctamente.");
        }));

        return armarPestana(arriba, tablaLibros, campos, botones);
    }

    // ---- Pestaña Usuarios

    /** Crea la pestaña de usuarios: buscador, tabla y formulario con botones. */
    private JPanel crearPestanaUsuarios() {
        JPanel arriba = new JPanel();
        arriba.add(new JLabel("Código:"));
        arriba.add(txtBuscarUsuario);
        arriba.add(boton("Buscar por código", () -> {
            Usuario usuario = buscarUsuario(numero(txtBuscarUsuario, "Código"));
            llenarUsuarios(Arrays.asList(usuario));
        }));
        arriba.add(boton("Mostrar todos", () -> actualizarTablas()));

        tablaUsuarios.getSelectionModel().addListSelectionListener(e -> {
            int fila = tablaUsuarios.getSelectedRow();
```

```
            if (fila >= 0) {
                txtUsuarioCodigo.setText(modeloUsuarios.getValueAt(fila, 0).toString());
                txtUsuarioNombre.setText(modeloUsuarios.getValueAt(fila, 1).toString());
            }
        });

        JPanel campos = new JPanel();
        campos.add(new JLabel("Código:"));
        campos.add(txtUsuarioCodigo);
        campos.add(new JLabel("Nombre:"));
        campos.add(txtUsuarioNombre);

        JPanel botones = new JPanel();
        botones.add(boton("Registrar", () -> {
            agregarUsuario(numero(txtUsuarioCodigo, "Código"), texto(txtUsuarioNombre, "Nombre"));
            limpiar(txtUsuarioCodigo, txtUsuarioNombre);
            cambioRealizado("Usuario registrado correctamente.");
        }));
        botones.add(boton("Modificar", () -> {
            modificarUsuario(numero(txtUsuarioCodigo, "Código"), texto(txtUsuarioNombre,
"Nombre"));
            limpiar(txtUsuarioCodigo, txtUsuarioNombre);
            cambioRealizado("Usuario modificado correctamente.");
        }));
        botones.add(boton("Eliminar", () -> {
            int codigo = numero(txtUsuarioCodigo, "Código");
            buscarUsuario(codigo); // verifica que exista
            if (!confirmar("¿Eliminar al usuario con código " + codigo + "?")) {
                return;
            }
            eliminarUsuario(codigo);
            limpiar(txtUsuarioCodigo, txtUsuarioNombre);
            cambioRealizado("Usuario eliminado correctamente.");
        }));

        return armarPestana(arriba, tablaUsuarios, campos, botones);
    }

    // ---- Pestaña Préstamos

    /**
     * Crea la pestaña de préstamos: filtro, tabla y botones para prestar, devolver, ver quién
tiene un libro y el historial.
     */
    private JPanel crearPestanaPrestamos() {
        JPanel arriba = new JPanel();
        chkActivos.addActionListener(e -> actualizarTablas());
        arriba.add(chkActivos);

        // Al hacer clic en un préstamo, sus códigos pasan al formulario (sirve para devolver)
        tablaPrestamos.getSelectionModel().addListSelectionListener(e -> {
            int fila = tablaPrestamos.getSelectedRow();
            if (fila >= 0) {
```

```
                txtPrestLibro.setText(modeloPrestamos.getValueAt(fila, 1).toString());
                txtPrestUsuario.setText(modeloPrestamos.getValueAt(fila, 3).toString());
            }
        });

        JPanel campos = new JPanel();
        campos.add(new JLabel("Código del libro:"));
        campos.add(txtPrestLibro);
        campos.add(new JLabel("Código del usuario:"));
        campos.add(txtPrestUsuario);

        JPanel botones = new JPanel();
        botones.add(boton("Registrar préstamo", () -> {
            realizarPrestamo(numero(txtPrestLibro, "Código del libro"), numero(txtPrestUsuario,
"Código del usuario"));
            limpiar(txtPrestLibro, txtPrestUsuario);
            cambioRealizado("Préstamo realizado correctamente.");
        }));
        botones.add(boton("Registrar devolución", () -> {
            int codigo = numero(txtPrestLibro, "Código del libro");
            prestamoActivoDeLibro(codigo); // verifica que esté prestado
            if (!confirmar("¿Registrar la devolución del libro " + codigo + "?")) {
                return;
            }
            devolverLibro(codigo);
            limpiar(txtPrestLibro, txtPrestUsuario);
            cambioRealizado("Libro devuelto correctamente.");
        }));
        botones.add(boton("¿Quién tiene el libro?", () -> {
            Prestamo p = prestamoActivoDeLibro(numero(txtPrestLibro, "Código del libro"));
            JOptionPane.showMessageDialog(this, "El libro \"" + p.getLibro().getTitulo() + "\" lo
tiene "
                    + p.getUsuario().getNombre() + " desde el " + p.getFecha() + ".");
        }));
        botones.add(boton("Historial del usuario", () ->
                llenarPrestamos(historialDeUsuario(numero(txtPrestUsuario, "Código del
usuario")))));

        return armarPestana(arriba, tablaPrestamos, campos, botones);
    }

    // ---- Métodos de apoyo de la ventana

    /** Junta las partes de una pestaña: arriba, tabla en el centro, y abajo los campos y los
botones. */
    private JPanel armarPestana(JPanel arriba, JTable tabla, JPanel campos, JPanel botones) {
        JPanel abajo = new JPanel(new GridLayout(2, 1));
        abajo.add(campos);
        abajo.add(botones);
        JPanel panel = new JPanel(new BorderLayout());
        panel.add(arriba, BorderLayout.NORTH);
        panel.add(new JScrollPane(tabla), BorderLayout.CENTER);
        panel.add(abajo, BorderLayout.SOUTH);
```

```
        return panel;
    }

    /** Crea el modelo de una tabla con las columnas indicadas, sin permitir editar las celdas a mano. */
    private static DefaultTableModel crearModelo(String... columnas) {
        return new DefaultTableModel(columnas, 0) {
            @Override
            public boolean isCellEditable(int fila, int columna) {
                return false;
            }
        };
    }

    /**
     * Crea un botón. Si la acción lanza un error, se muestra en una ventana y el programa sigue funcionando.
     *
     * @param texto texto del botón
     * @param accion lo que hace el botón al pulsarlo
     */
    private JButton boton(String texto, Accion accion) {
        JButton boton = new JButton(texto);
        boton.addActionListener(e -> {
            try {
                accion.ejecutar();
            } catch (Exception ex) {
                mostrarError(ex.getMessage());
            }
        });
        return boton;
    }

    /**
     * Lee un número de un campo de texto (validación de entrada).
     *
     * @param campo campo de texto a leer
     * @param nombre nombre del campo, para el mensaje de error
     * @return el número escrito
     * @throws EntradaInvalidaException si no es un número válido o no es mayor que cero
     */
    private int numero(JTextField campo, String nombre) throws EntradaInvalidaException {
        try {
            int valor = Integer.parseInt(campo.getText().trim());
            if (valor <= 0) {
                throw new EntradaInvalidaException("El campo \"" + nombre + "\" debe ser mayor que cero.");
            }
            return valor;
        } catch (NumberFormatException e) {
            throw new EntradaInvalidaException("El campo \"" + nombre + "\" debe ser un número válido.");
        }
```

```
    }

    /**
     * Lee un texto obligatorio de un campo.
     *
     * @return el texto sin espacios sobrantes
     * @throws EntradaInvalidaException si el campo está vacío
     */
    private String texto(JTextField campo, String nombre) throws EntradaInvalidaException {
        String texto = campo.getText().trim();
        if (texto.isEmpty()) {
            throw new EntradaInvalidaException("El campo \"" + nombre + "\" es obligatorio.");
        }
        return texto;
    }

    /** Deja vacíos los campos de texto indicados. */
    private void limpiar(JTextField... campos) {
        for (JTextField c : campos) {
            c.setText("");
        }
    }

    /** Muestra una ventana de error con el mensaje. */
    private void mostrarError(String mensaje) {
        JOptionPane.showMessageDialog(this, mensaje, "Error", JOptionPane.ERROR_MESSAGE);
    }

    /**
     * Pregunta Sí o No al usuario.
     *
     * @return true si eligió Sí
     */
    private boolean confirmar(String pregunta) {
        return JOptionPane.showConfirmDialog(this, pregunta, "Confirmar",
                JOptionPane.YES_NO_OPTION) == JOptionPane.YES_OPTION;
    }

    /**
     * Se llama después de registrar, modificar, eliminar, prestar o devolver:
     * refresca las tablas, guarda los datos en el archivo y avisa al usuario.
     */
    private void cambioRealizado(String mensaje) {
        actualizarTablas();
        guardar();
        JOptionPane.showMessageDialog(this, mensaje);
    }

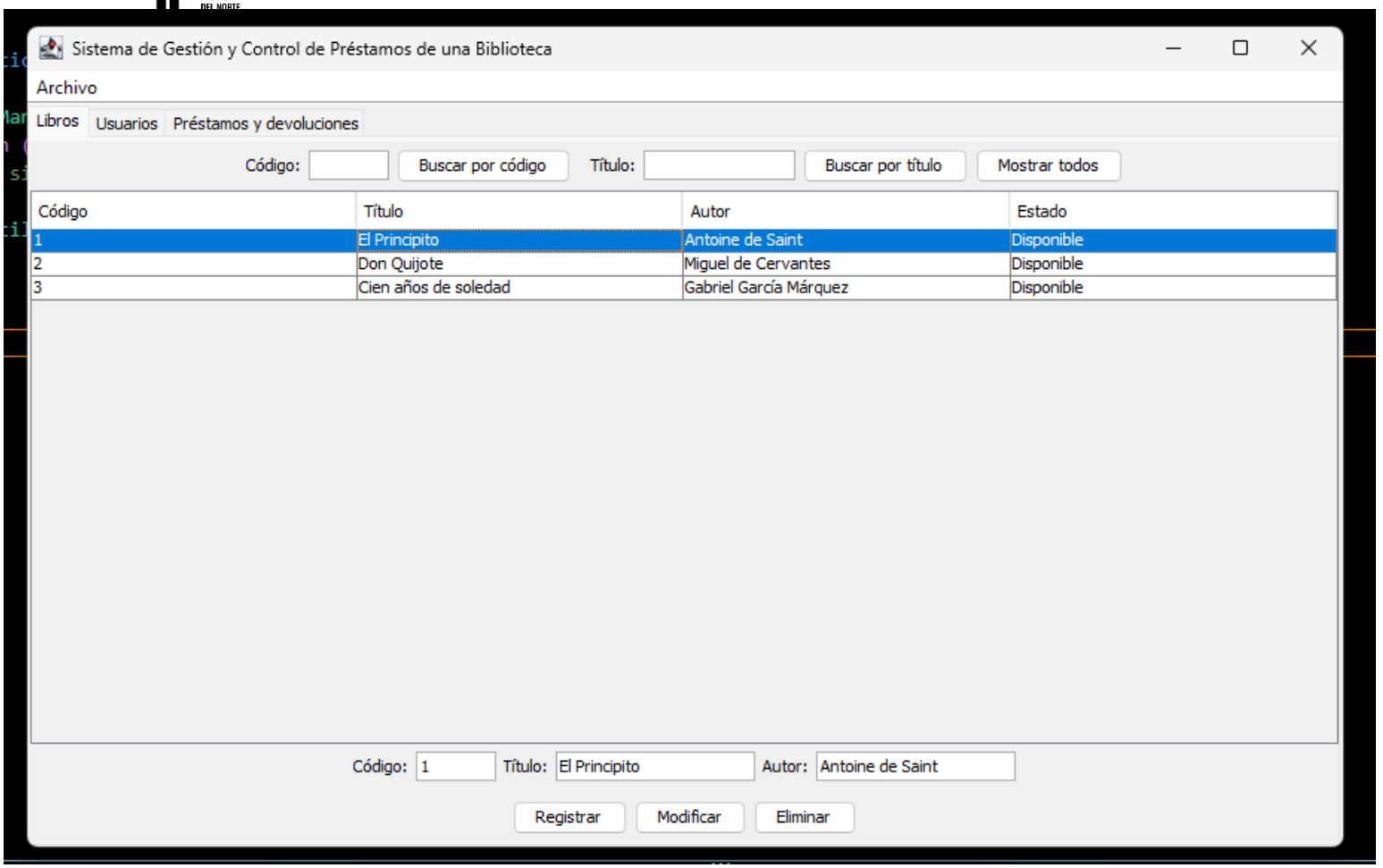
    /** Vuelve a llenar las tres tablas con los datos actuales. */
    private void actualizarTablas() {
        llenarLibros(libros);
        llenarUsuarios(usuarios);
        llenarPrestamos(chkActivos.isSelected() ? listarPrestamosActivos() : prestamos);
```

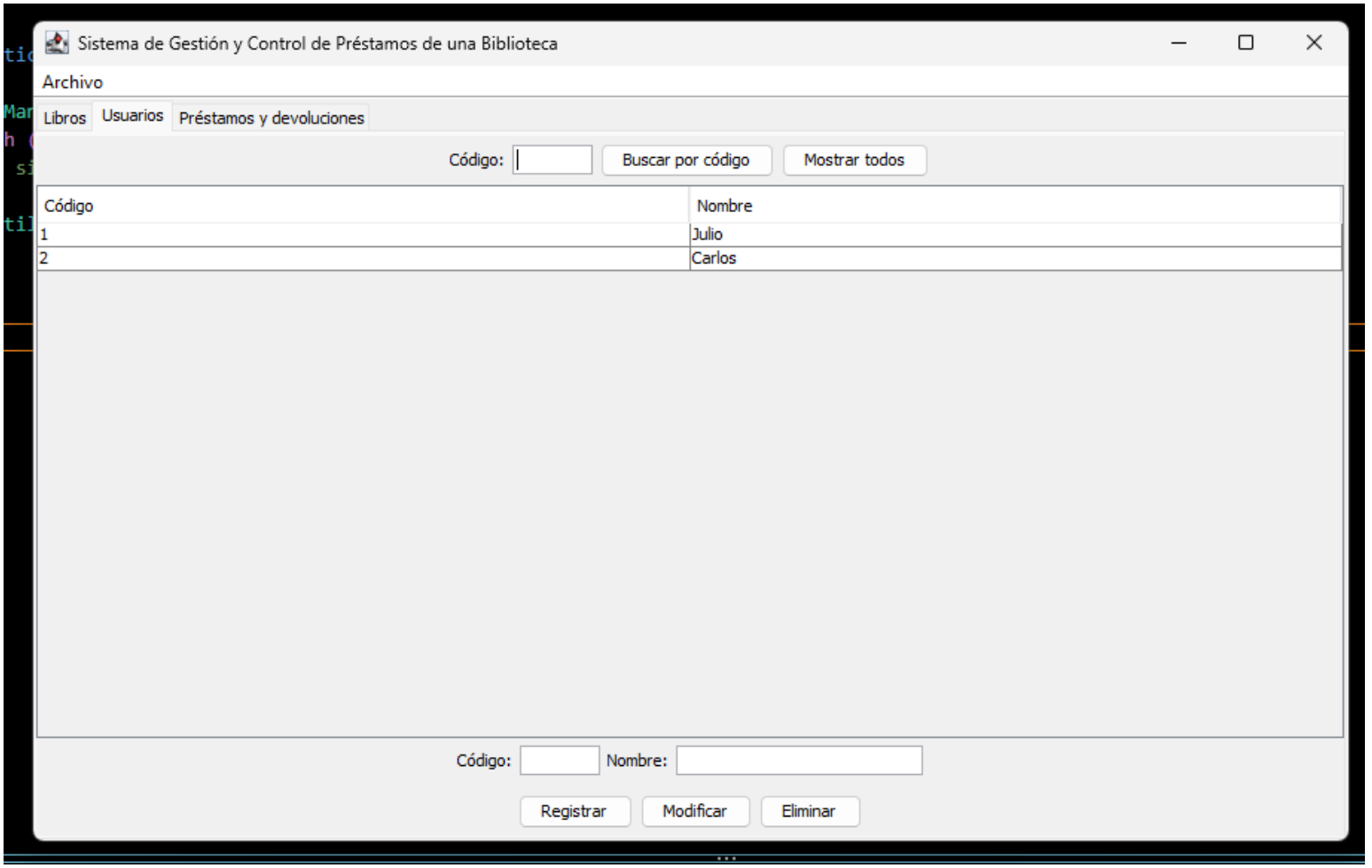
```
    }

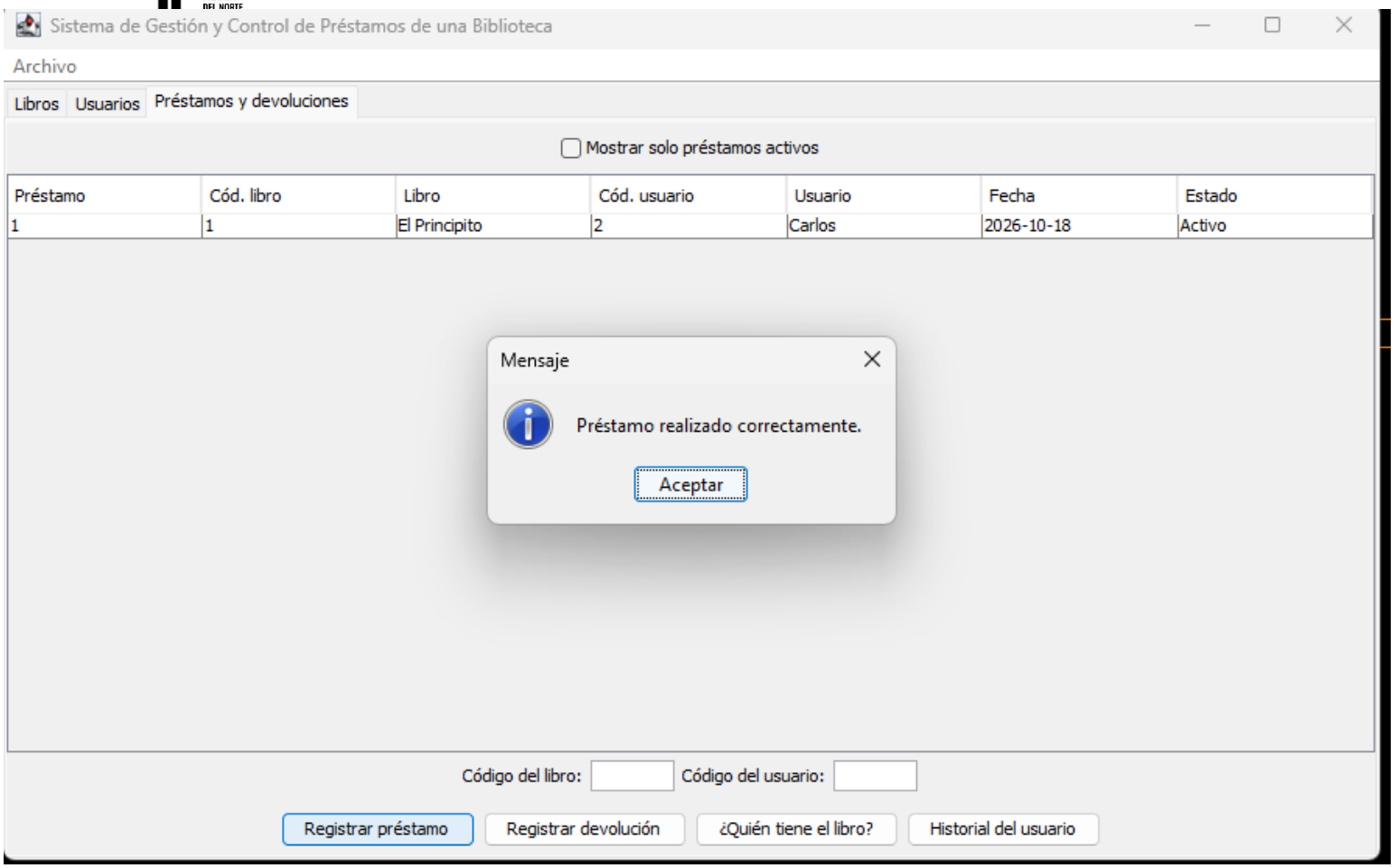
    /** Borra las filas de la tabla de libros y la llena con la lista recibida. */
    private void llenarLibros(List<Libro> lista) {
        modeloLibros.setRowCount(0);
        for (Libro l : lista) {
            modeloLibros.addRow(new Object[]{l.getCodigo(), l.getTitulo(), l.getAutor(),
l.getEstado()});
        }
    }

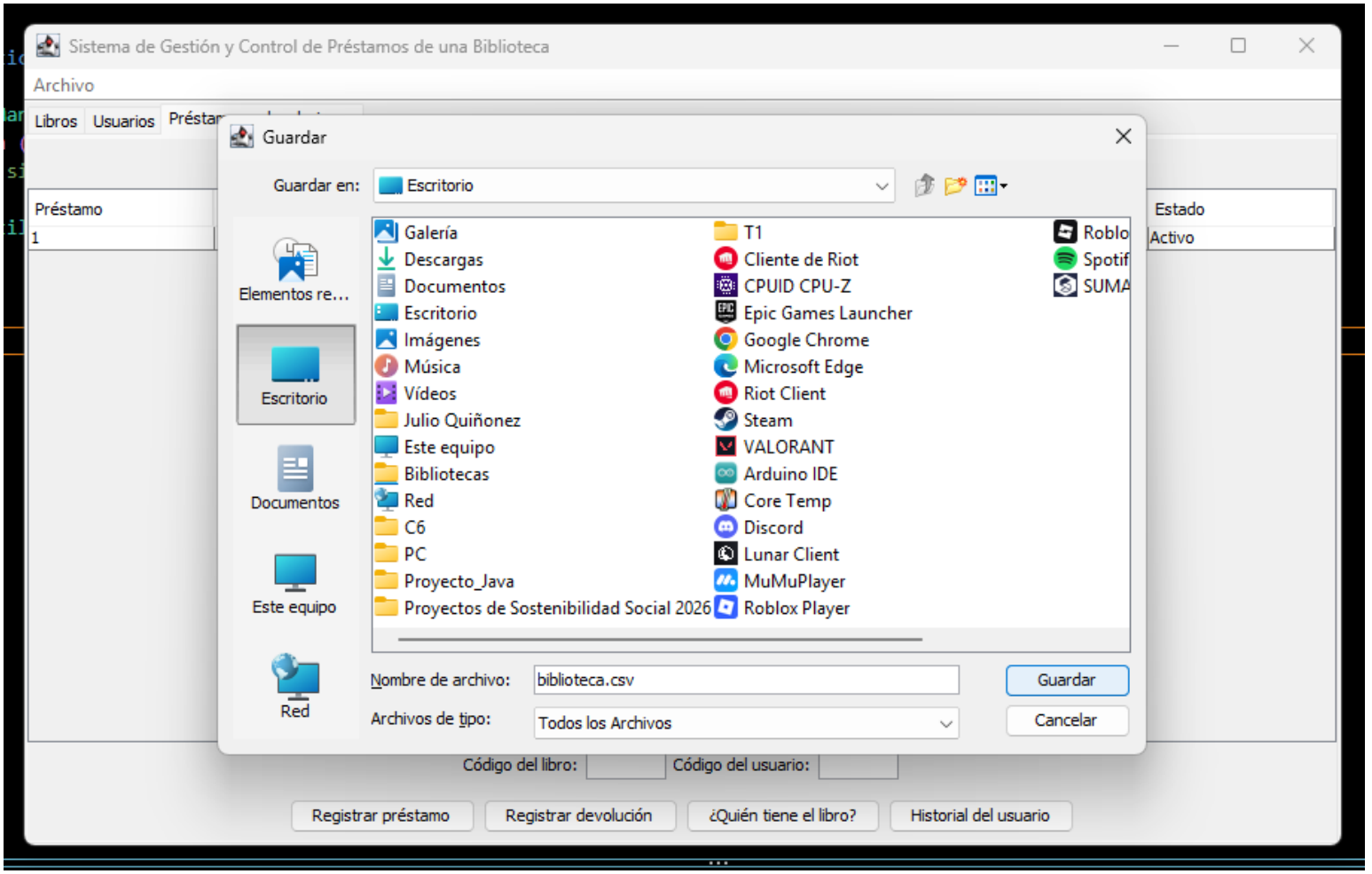
    /** Borra las filas de la tabla de usuarios y la llena con la lista recibida. */
    private void llenarUsuarios(List<Usuario> lista) {
        modeloUsuarios.setRowCount(0);
        for (Usuario u : lista) {
            modeloUsuarios.addRow(new Object[]{u.getCodigo(), u.getNombre()});
        }
    }

    /** Borra las filas de la tabla de préstamos y la llena con la lista recibida. */
    private void llenarPrestamos(List<Prestamo> lista) {
        modeloPrestamos.setRowCount(0);
        for (Prestamo p : lista) {
            modeloPrestamos.addRow(new Object[]{p.getCodigo(), p.getLibro().getCodigo(),
p.getLibro().getTitulo(),
                    p.getUsuario().getCodigo(), p.getUsuario().getNombre(), p.getFecha(),
p.getEstado()});
        }
    }
}
```


Sistema de Gestión y Control de Préstamos de una Biblioteca
Archivo
Libros
Usuarios
Préstamos y devoluciones
Código:
Buscar por código
Título:
Buscar por título
Mostrar todos
Código
Título
Autor
Estado
1
El Principito
Antoine de Saint
Disponible
2
Don Quijote
Miguel de Cervantes
Disponible
3
Cien años de soledad
Gabriel García Márquez
Disponible
Código: 1
Título: El Principito
Autor: Antoine de Saint
Registrar
Modificar
Eliminar


Sistema de Gestión y Control de Préstamos de una Biblioteca
Archivo
Libros
Usuarios
Préstamos y devoluciones
Código:
Buscar por código
Mostrar todos
Código
Nombre
1
Julio
2
Carlos
Código:
Nombre:
Registrar
Modificar
Eliminar


Sistema de Gestión y Control de Préstamos de una Biblioteca
Archivo
Libros
Usuarios
Préstamos y devoluciones
Mostrar solo préstamos activos
Préstamo
Cód. libro
Libro
Cód. usuario
Usuario
Fecha
Estado
1
1
El Principito
2
Carlos
2026-10-18
Activo
Mensaje
Préstamo realizado correctamente.
Aceptar
Código del libro:
Código del usuario:
Registrar préstamo
Registrar devolución
¿Quién tiene el libro?
Historial del usuario


Sistema de Gestión y Control de Préstamos de una Biblioteca
Archivo
Libros
Usuarios
Guardar
Guardar en:
Escritorio
Elementos re...
Escritorio
Documentos
Este equipo
Red
Galería
Descargas
Documentos
Escritorio
Imágenes
Música
Vídeos
Julio Quiñonez
Este equipo
Bibliotecas
Red
C6
PC
Proyecto_Java
Proyectos de Sostenibilidad Social 2026
T1
Cliente de Riot
CPUID CPU-Z
Epic Games Launcher
Google Chrome
Microsoft Edge
Riot Client
Steam
VALORANT
Arduino IDE
Core Temp
Discord
Lunar Client
MuMuPlayer
Roblox Player
Nombre de archivo:
biblioteca.csv
Archivos de tipo:
Todos los Archivos
Guardar
Cancelar
Préstamo
Estado
1
Activo
Código del libro:
Código del usuario:
Registrar préstamo
Registrar devolución
¿Quién tiene el libro?
Historial del usuario

Breve Explicación:

El programa es un sistema para controlar los libros, los usuarios y los préstamos de una biblioteca, hecho en Java con una ventana gráfica. El código está organizado en tres partes: la clase principal `App`, la carpeta `sistema` con las cuatro clases que hacen funcionar el programa, y la carpeta `excepciones` con los errores propios del sistema.

`App` es la clase que se ejecuta. Su único trabajo es abrir la ventana principal del sistema.

La carpeta `sistema` tiene cuatro clases. `Libro` guarda el código, el título, el autor y si el libro está disponible o prestado. `Usuario` guarda el código y el nombre del lector. `Prestamo` relaciona un libro con un usuario y guarda la fecha del préstamo y si ya fue devuelto. `Biblioteca` es la clase central: guarda las listas de libros, usuarios y préstamos, tiene todas las operaciones (registrar, buscar, modificar, eliminar, prestar y devolver), maneja el archivo de datos y además es la ventana con la que el usuario trabaja.

La ventana tiene tres pestañas: Libros, Usuarios y Préstamos. En cada una hay una tabla con los datos y un formulario con botones. Al registrar o prestar, el programa revisa que los campos no estén vacíos y que los códigos sean números. Si algo está mal, por ejemplo un código repetido o un libro que ya está prestado, muestra un mensaje de error y sigue funcionando. Esto se logra con las excepciones propias del sistema, que se lanzan cuando se rompe una regla y se muestran al usuario en una ventana.

Después de cada cambio, el programa guarda las tres listas en un archivo llamado `datos.dat`, usando serialización de objetos de Java, que consiste en convertir los objetos en bytes para escribirlos en el disco. Cuando el programa se vuelve a abrir, lee ese archivo y recupera todo tal como estaba. Si el archivo no existe, empieza con unos datos de ejemplo.

Además, desde el menú Archivo se puede exportar toda la información a un archivo con extensión `.csv`, que se abre con Excel y muestra tres bloques: libros, usuarios y préstamos. Así los datos se pueden consultar o imprimir fuera del programa.

Para buscar libros, la clase `Biblioteca` tiene dos métodos con el mismo nombre, `buscarLibro`: uno recibe un código y otro recibe un título. Java sabe cuál usar según el tipo de dato que recibe, y a esto se le llama sobrecarga de métodos. La búsqueda por título no distingue mayúsculas de minúsculas, por lo que escribir “don quijote” encuentra “Don Quijote”.

Referencias

Ishikawa, K. (1986). *Guide to quality control* (2.ª ed. rev.). Asian Productivity Organization.

Matamoros Motoche, M. A., y Sánchez Idrovo, M. F. (2011). *Análisis, diseño y creación de una aplicación web para el control de préstamos y consultas en la Biblioteca de la Municipalidad del cantón Biblián* [Tesis de grado, Universidad Politécnica Salesiana]. Repositorio Institucional UPS.

https://dspace.ups.edu.ec/handle/123456789/2103

Quispe-Lukaña, D. A., y Ulloa-Gallardo, N. J. (2024). Desarrollo e implementación de un sistema web para optimizar los procesos de préstamo de libros de una biblioteca, Tambopata. *Revista Amazonía Digital, 3*(2), e288. https://revistas.unamad.edu.pe/index.php/rad/article/view/288

Ruiz Loro, C. E. (2021). *Diseño de un sistema web con software libre para la gestión de biblioteca en la institución educativa Federico Villarreal - Piura; 2018* [Trabajo de investigación, Universidad Católica Los Ángeles de Chimbote]. Repositorio Institucional ULADECH. https://hdl.handle.net/20.500.13032/22813